

UCS503P Software Engineering Lab: Project Report

CampusSwap

Room-Exchange and Slot-Coordination System for Campus

Prototype-stage report: multi-way hostel room swaps with Top Trading Cycles

Submitted by:

1024240152 Jahnavi

1024240102 Ronit Saini

1024240001 Ranvir

1024240010 Krishna Kapoor

BE Third Year, 2026-27

Submitted to:

Dr. Raghav B. Venkataramaiyer

Mid-Semester Evaluation (Prototype Stage)

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Motivation	5
1.2	Problem Statement	5
1.3	Objectives	6
1.4	Scope of the Prototype	6
2	Requirements and Design	7
2.1	Users and Use Cases	7
2.2	Functional Requirements	8
2.3	Non-Functional Requirements	8
2.4	System Architecture	9
2.5	Data Model	9
2.6	Behaviour	10
3	Implementation	13
3.1	Matching Engine	13
3.1.1	Model	13
3.1.2	Top Trading Cycles	13
3.1.3	Baseline and Metric	14
3.2	Request and Confirmation Workflow	14
3.3	Security	14
3.4	User Interface	15
3.5	Repository, CI and Documentation	19
4	Testing and Evaluation	21
4.1	Testing	21
4.2	Match Yield	21
4.3	Performance	22
4.4	Objectives Review	23
4.5	Deviations from the Proposal	23
5	Conclusion and Future Work	25
5.1	Summary	25
5.2	Key Findings	25
5.3	Future Work	25
5.4	Team Roles	26
	Bibliography	27

Introduction

1.1 Background and Motivation

Hostel rooms at a university are allocated once, at the start of a semester, and are then rarely changed. Students who would like to move, for example to a lower floor, a single room, or closer to friends, can only do so by *trading* with another student, because hostels run close to full capacity. Today these trades are arranged informally through WhatsApp groups, notice boards and personal contacts, and are approved on paper by the hostel office.

This project, **CampusSwap**, replaces that informal process with a software workflow. Students submit a ranked list of rooms they would prefer; the system finds *chains* of exchanges in which every participant moves to a room they prefer; every participant confirms; and the warden approves. The matching step uses the **Top Trading Cycles** (TTC) algorithm of Shapley and Scarf [SS74], which is the standard method for markets in which each participant already owns one indivisible good [AS99].

1.2 Problem Statement

The current process has four bottlenecks, identified in our proposal:

- **Fragmented information:** requests are scattered across chat groups and notice boards, so open offers are hard to find.
- **Search friction:** people look only for direct 1:1 swaps (A with B). Multi-way chains such as $A \rightarrow B \rightarrow C \rightarrow A$ satisfy everyone without needing an empty room, but are almost impossible to find by hand.
- **Administrative overhead:** approved swaps need paper applications and manual approval, with no central record.
- **No visibility:** applicants cannot tell whether a match is pending, being processed, or impossible.

Finding such chains is a graph-search problem, which makes it well suited to a software solution.

1.3 Objectives

The proposal defined Phase 1 success as *accepting a request, persisting it to the database, executing a match, and rendering the result*. The objectives for this prototype stage were:

1. **O1 – Data model:** a relational schema for hostels, rooms, beds, students, requests and matches.
2. **O2 – Student module:** authentication and ranked preference entry (up to five rooms) with full validation.
3. **O3 – Matching engine:** a TTC implementation backed by unit tests on synthetic data.
4. **O4 – Dashboards and CI:** a student status dashboard, an administrator console, and continuous integration that runs the tests on every change.
5. **O5 – Evaluation:** measure Match Yield (MY), the percentage of students placed in an exchange chain, against the target $MY \geq 60\%$ for 100 requests, and compare with direct 1:1 swaps only.

Chapter 4 reports whether each objective was met.

1.4 Scope of the Prototype

Included: student and hostel-staff login; room browsing and ranked requests (feature 1); TTC matching rounds with a dry-run mode and a TTC-versus-1:1 comparison (feature 2); the multi-party confirmation workflow with a 48-hour deadline, which the proposal planned for Phase 2; warden approval; CSV export; an audit log.

Not included yet: the institute single sign-on, e-mail notifications, the PDF sign-off form, the faculty slot-exchange module, and deployment to a public server. All data used in this report is synthetic.

Requirements and Design

2.1 Users and Use Cases

Two kinds of people use the system: **students**, who submit, withdraw, confirm or decline swaps, and **hostel staff (the warden)**, who run matching rounds and approve swaps. Figure 2.1 shows the use cases.

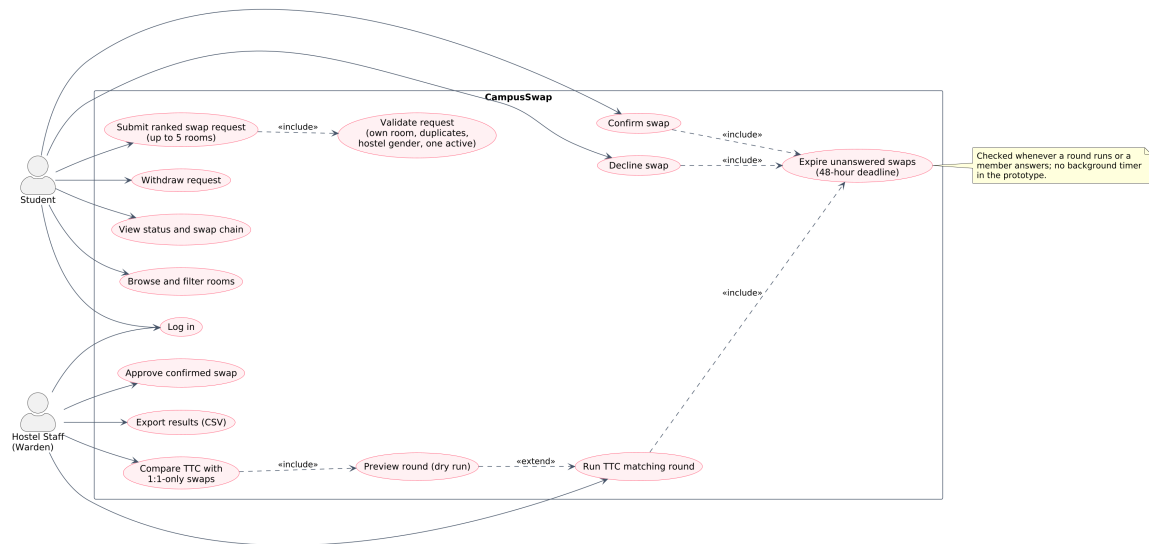


Figure 2.1: Use Case diagram of the prototype.

The relationships in the diagram are justified as follows:

- *Submit ranked request* $\langle\langle\text{include}\rangle\rangle$ *Validate request*: validation is a mandatory part of every submission.
- *Run matching round*, *Confirm swap* and *Decline swap* $\langle\langle\text{include}\rangle\rangle$ *Expire unanswered swaps*: each of them first dissolves chains whose 48-hour deadline has passed.
- *Preview round (dry run)* $\langle\langle\text{extend}\rangle\rangle$ *Run matching round*: an optional variation chosen by the warden.

- *Compare TTC with 1:1 swaps* `<<include>>` *Preview round*: a comparison always runs both algorithms as dry runs.

There is no clock actor: in the prototype the deadline is checked whenever something happens, not by a background timer.

2.2 Functional Requirements

ID	Requirement
FR1	Students and staff log in with an institute e-mail and password; each role sees only its own pages and API endpoints.
FR2	A student can browse rooms only in hostels they are allowed to live in, filtered by hostel, floor, type, AC and washroom.
FR3	A student submits 1–5 ranked rooms; the system rejects duplicates, their own room, unknown rooms, the other gender’s hostels, and a second active request. Each request receives a tracking ID.
FR4	A student can withdraw a request while it is still in the pool.
FR5	The warden runs a TTC round over all open requests; the result is stored with a readable round-by-round trace.
FR6	The warden can preview a round without saving it, and compare TTC with direct 1:1 swaps on the same pool.
FR7	Every member of a proposed chain confirms or declines within 48 hours; one decline or a missed deadline cancels the chain and returns the others to the pool.
FR8	The warden approves a fully confirmed chain, after which the students’ beds change hands.
FR9	Every request, round, answer and approval is recorded in an audit log; all requests can be exported as CSV.

Table 2.1: Functional requirements.

2.3 Non-Functional Requirements

- **Correctness**: every matching round is saved in one database transaction (all or nothing).
- **Security**: salted password hashes; session tokens in HttpOnly cookies with only their hash stored; server-side role checks; escaping of all user-supplied text in the pages.

- **Responsiveness:** pages work on phones and desktops.
- **Performance:** a matching round for a whole hostel completes well within one second.
- **Operability:** the prototype runs with one command and without internet access (fonts and styles are bundled).

2.4 System Architecture

CampusSwap is a three-tier web application with a separate matching engine (Figure 2.3 shows the tiers in action):

1. **Presentation:** four HTML pages with small JavaScript modules that call a JSON API.
2. **Application:** FastAPI routes [Ram26] that check the caller’s session and role, and *services* that contain every business rule.
3. **Data:** a relational SQLite database accessed through SQLAlchemy [SQL26].
4. **Engine:** a pure-Python TTC module with no database or web code, called by the matching service. It can therefore be unit-tested in isolation and reused for faculty slots later.

Layer	Technology
Frontend	HTML, JavaScript modules, Tailwind CSS 3.4 (built locally)
Backend	Python 3.12, FastAPI 0.141, Uvicorn
Database	SQLite through SQLAlchemy 2.0 (ACID transactions)
Testing and CI	pytest 9 (90 tests), GitHub Actions
Diagrams and docs	PlantUML [Pla26], MkDocs on GitHub Pages

Table 2.2: Technology stack.

2.5 Data Model

Figure 2.2 shows the classes. Key design decisions:

- **Generalisation:** `Student` and `Admin` are kinds of `User`; both are stored in one table.

- **Composition:** a **Hostel** owns its **Rooms** and a room owns its 1–3 **Seats** (beds); a **SwapRequest** owns its 1–5 **Preferences**; a **MatchRun** owns its **SwapCycles**, each with at least two **CycleMembers**.
- **Association:** a seat has at most one occupant and a student at most one seat. Students exist independently of seats, so this is an association rather than composition.
- **Why seats and not rooms:** in a double room only one of the two students moves, so the unit of exchange is a bed.

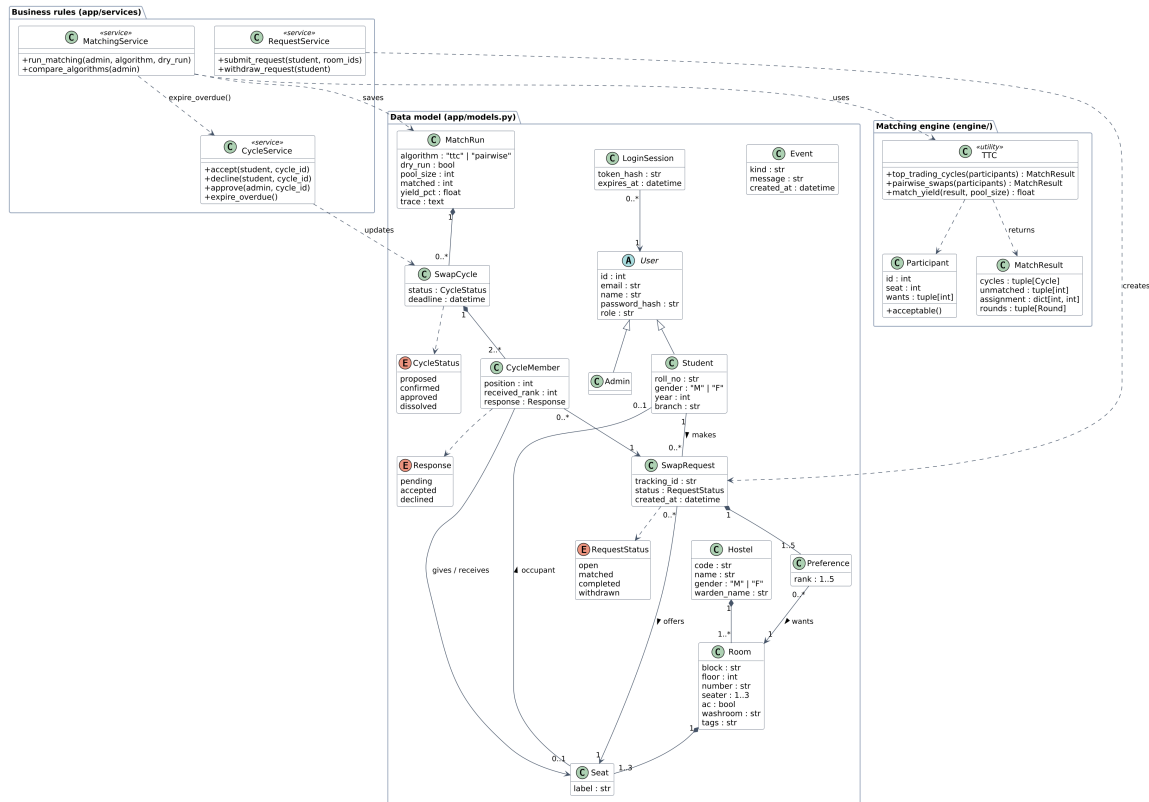


Figure 2.2: Class diagram: data model, services and engine.

2.6 Behaviour

Figure 2.3 traces one swap from request to approval through every layer, and Figure 2.4 shows the life of a request, including the decline and deadline branches.

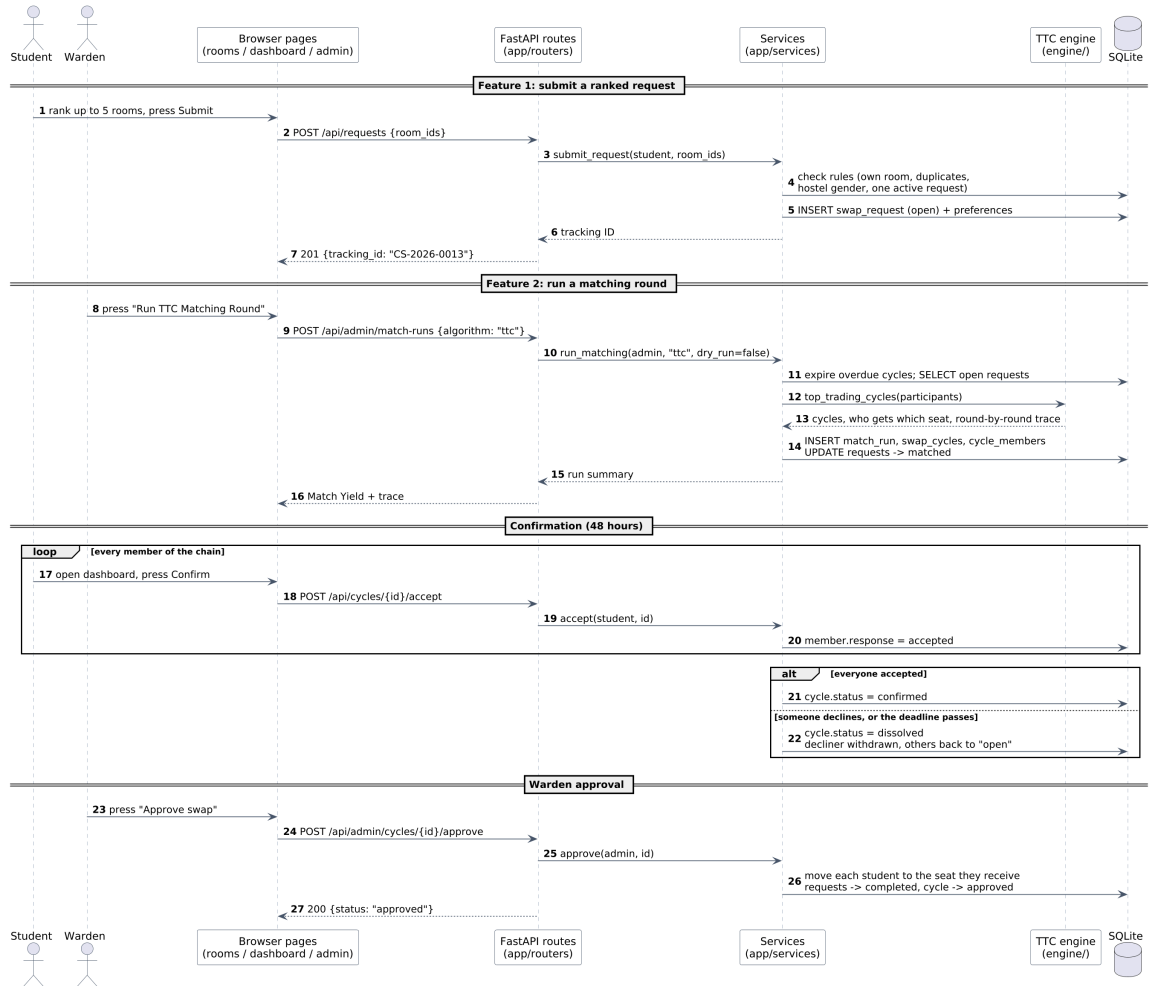


Figure 2.3: Sequence diagram: submit, match, confirm and approve.

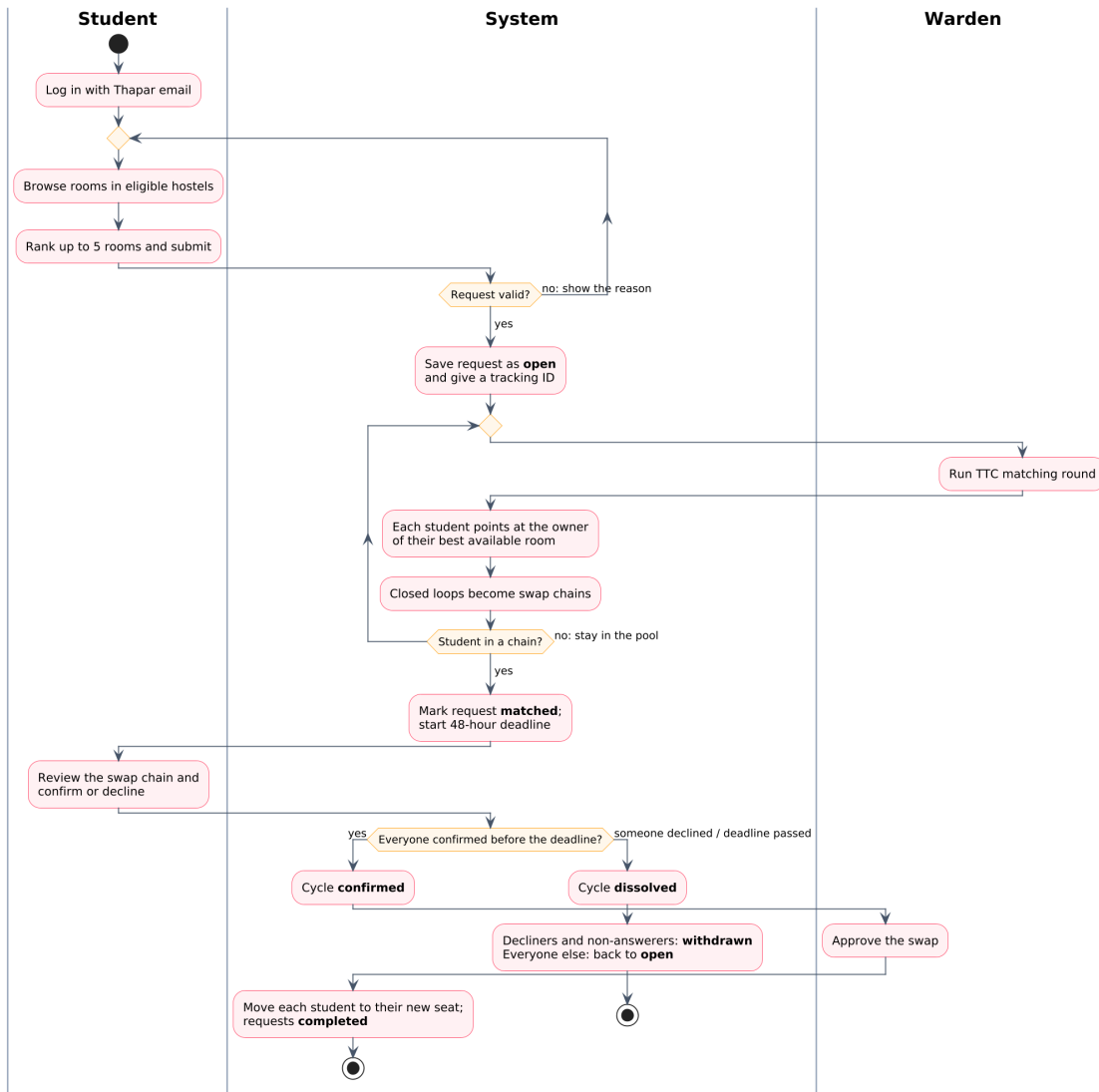


Figure 2.4: Activity diagram: the life of a swap request.

Implementation

3.1 Matching Engine

3.1.1 Model

Each open request becomes a *participant* p that owns one seat s_p and has a ranked list w_p of seats it would accept. Students rank *rooms*; the service expands each room into its seats in seat-ID order, so a double room can be obtained from either occupant who is in the pool. Seats listed after the participant's own seat count as worse than staying.

3.1.2 Top Trading Cycles

Algorithm 1 is implemented in `code/backend/engine/ttc.py`.

Algorithm 1 Top Trading Cycles as implemented in CampusSwap

Require: participants P ; each $p \in P$ owns seat s_p and ranks seats w_p

```

1:  $A \leftarrow P$  ▷ participants still in play
2: while  $A \neq \emptyset$  do
3:   for each  $p \in A$  whose current target has left do
4:      $target(p) \leftarrow$  owner of the first seat in  $w_p$  still held in  $A$ , or  $p$  itself
5:   end for
6:   find every cycle in the graph  $p \rightarrow target(p)$ 
7:   for each cycle  $C$  do
8:     if  $|C| = 1$  then
9:       the member keeps its own seat (unmatched)
10:    else
11:      each  $p \in C$  receives the seat of  $target(p)$ 
12:    end if
13:     $A \leftarrow A \setminus C$ 
14:  end for
15: end while

```

Because every participant points at exactly one participant, the pointer graph always contains at least one cycle, so each round removes at least one participant. A pointer only moves forward in its list, because a taken seat never returns; it is recomputed only when its target leaves. Cycles are found in ID order and rotated to start at their smallest ID, so the result does not depend on the order of the input.

Guarantees. TTC never moves a student to a seat they ranked below their own (individual rationality); its result is Pareto efficient and in the core [SS74; RP77]; and truthful ranking is a dominant strategy [Rot82].

Complexity. Finding cycles is linear in the number of participants per round and there are at most n rounds, giving $O(n^2)$ in the worst case; pointer advances add up to the total length of all preference lists.

3.1.3 Baseline and Metric

For the evaluation, `pairwise_swaps()` pairs two students when each ranked the other's seat, taking the best combined rank first and allowing each student in at most one pair. It is only ever run as a dry run. Match Yield is

$$\text{MY} = \frac{\text{students placed in a chain}}{\text{students in the pool}} \times 100.$$

3.2 Request and Confirmation Workflow

All rules live in service modules (`app/services`); the HTTP routes only parse input, call a service and commit. A rule violation raises a `DomainError` that the API returns as JSON with a short error code (for example `own_room`).

From	To	When
–	open	the student submits a valid ranked list
open	withdrawn	the student withdraws
open	matched	a saved TTC round places the request in a chain
matched	open	the chain is dissolved but this student had not declined
matched	withdrawn	the student declined, or did not answer before the deadline
matched	completed	the warden approves the confirmed chain

Table 3.1: Status changes of a swap request.

On approval, every seat in the chain is first emptied and then refilled, because the database allows only one occupant per seat.

3.3 Security

- Passwords are stored as salted `scrypt` hashes and compared in constant time.

- Login creates a random token sent in an HttpOnly, SameSite=Lax cookie; only its SHA-256 hash is stored, and sessions expire after 12 hours. Unknown e-mails and wrong passwords give the same answer.
- Every administrator endpoint checks the role on the server.
- The pages escape all text received from the server before inserting it, preventing cross-site scripting.
- Only one matching round can run at a time, and each round is a single transaction.

3.4 User Interface

The four pages follow reference designs chosen by the team. Content in the references that was fictitious for this project (other algorithm names, invented statistics, security features we do not have) was replaced with live data from the API. Figures 3.1–3.6 show the prototype.

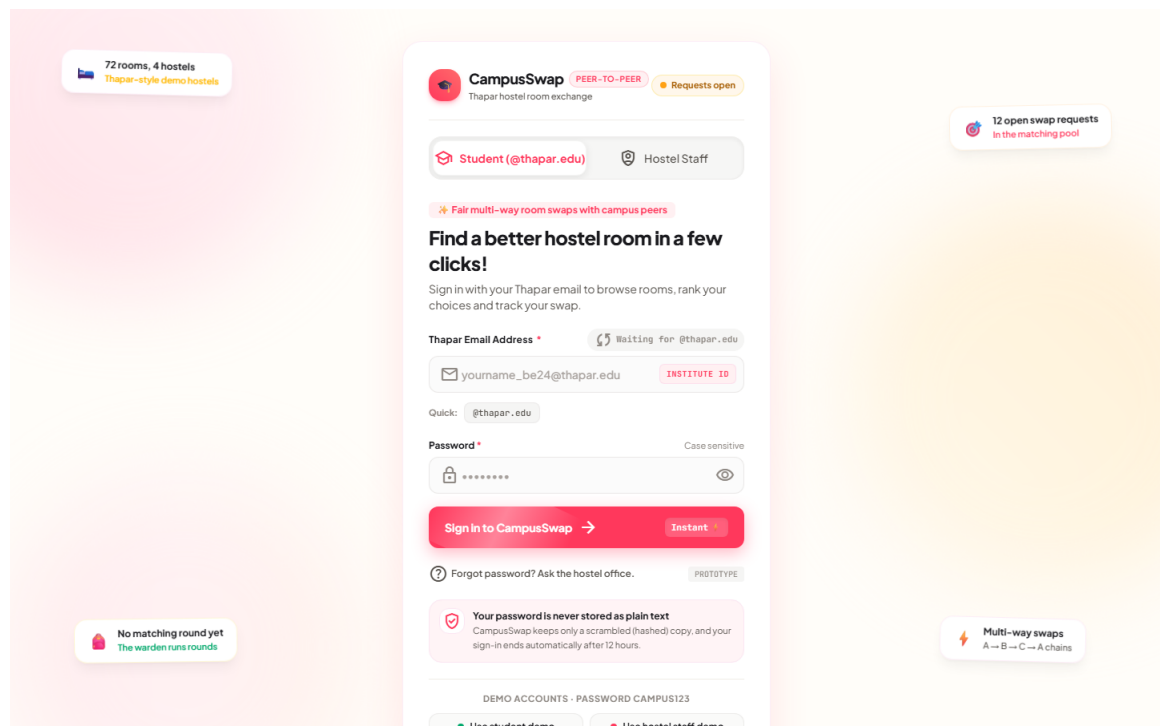


Figure 3.1: Login page with student / hostel staff switch and live pool statistics.

16 UCS503P Software Engineering Lab: Project Report

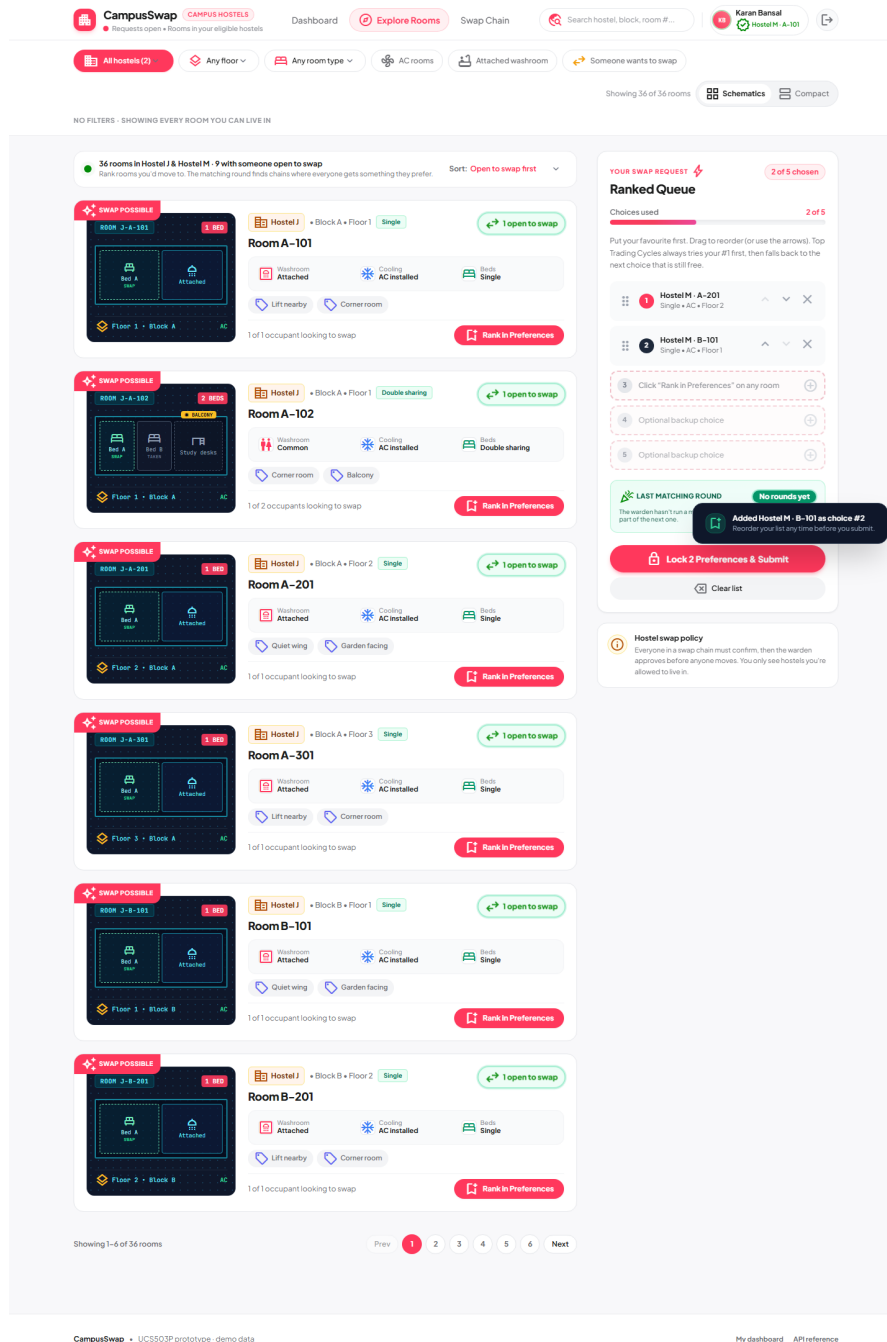


Figure 3.2: Feature 1: exploring rooms and building a ranked list.

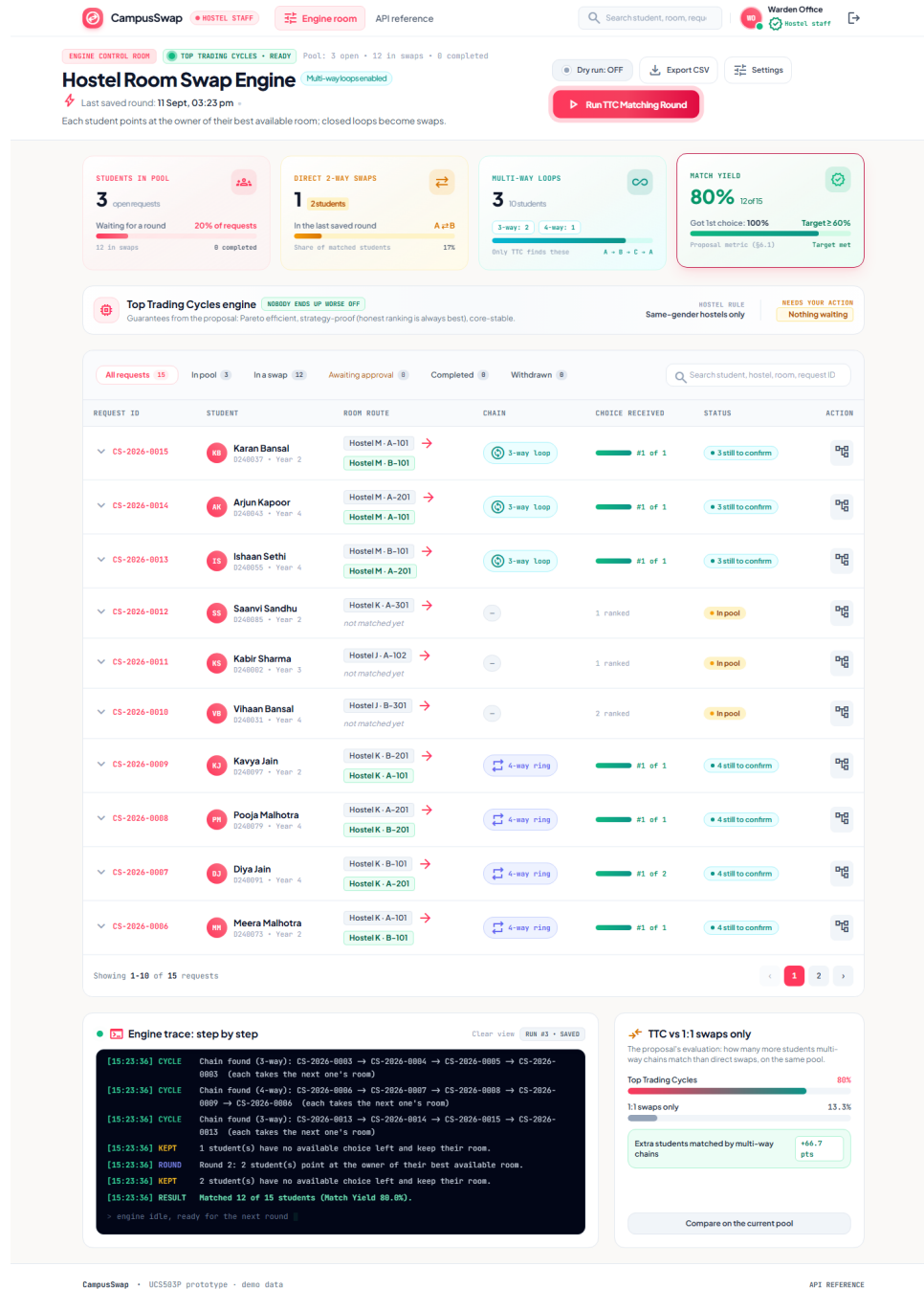


Figure 3.3: Feature 2: the administrator's engine room after a saved TTC round.

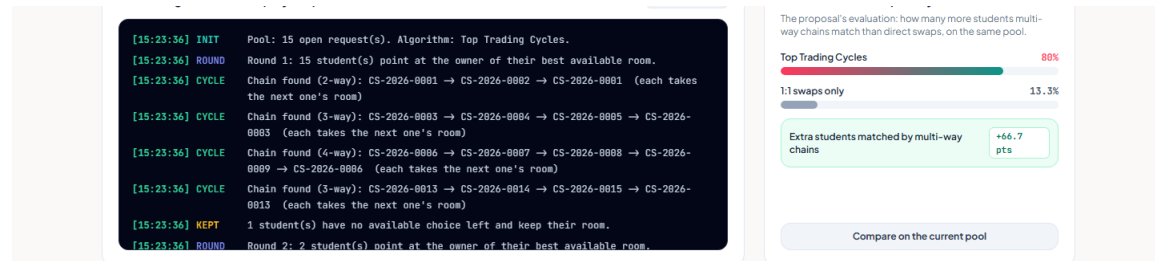


Figure 3.4: The engine trace explains each TTC round in plain words.

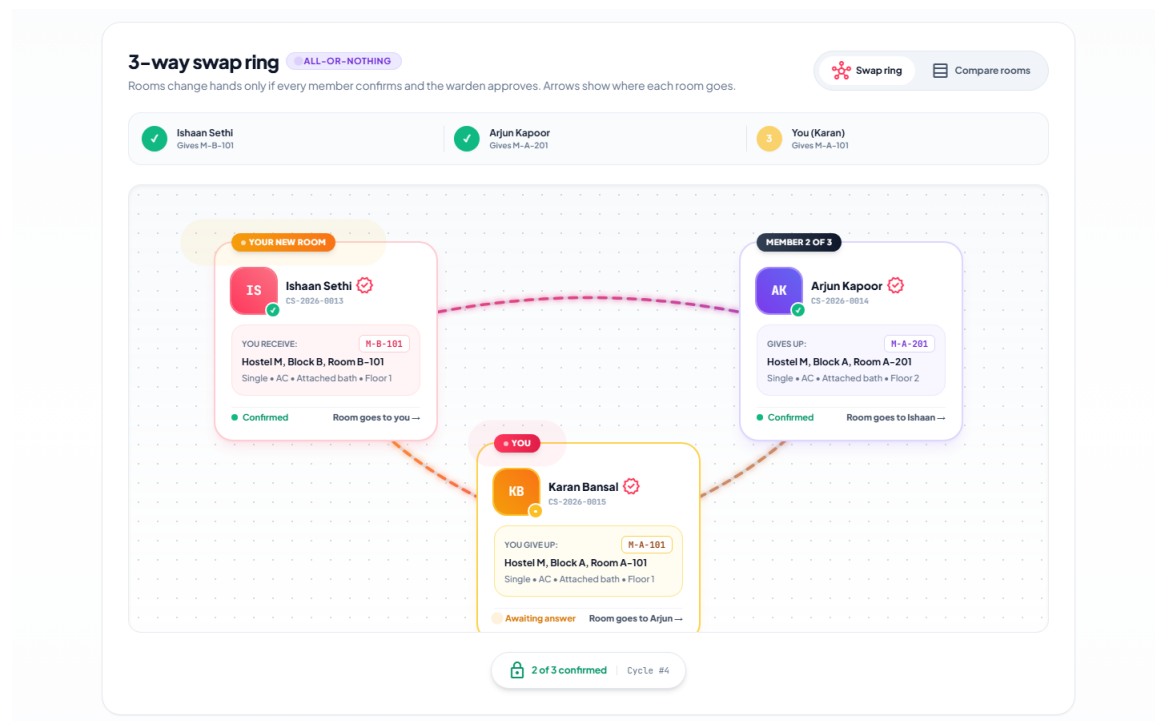


Figure 3.5: Student dashboard: a 3-way swap chain awaiting confirmation.

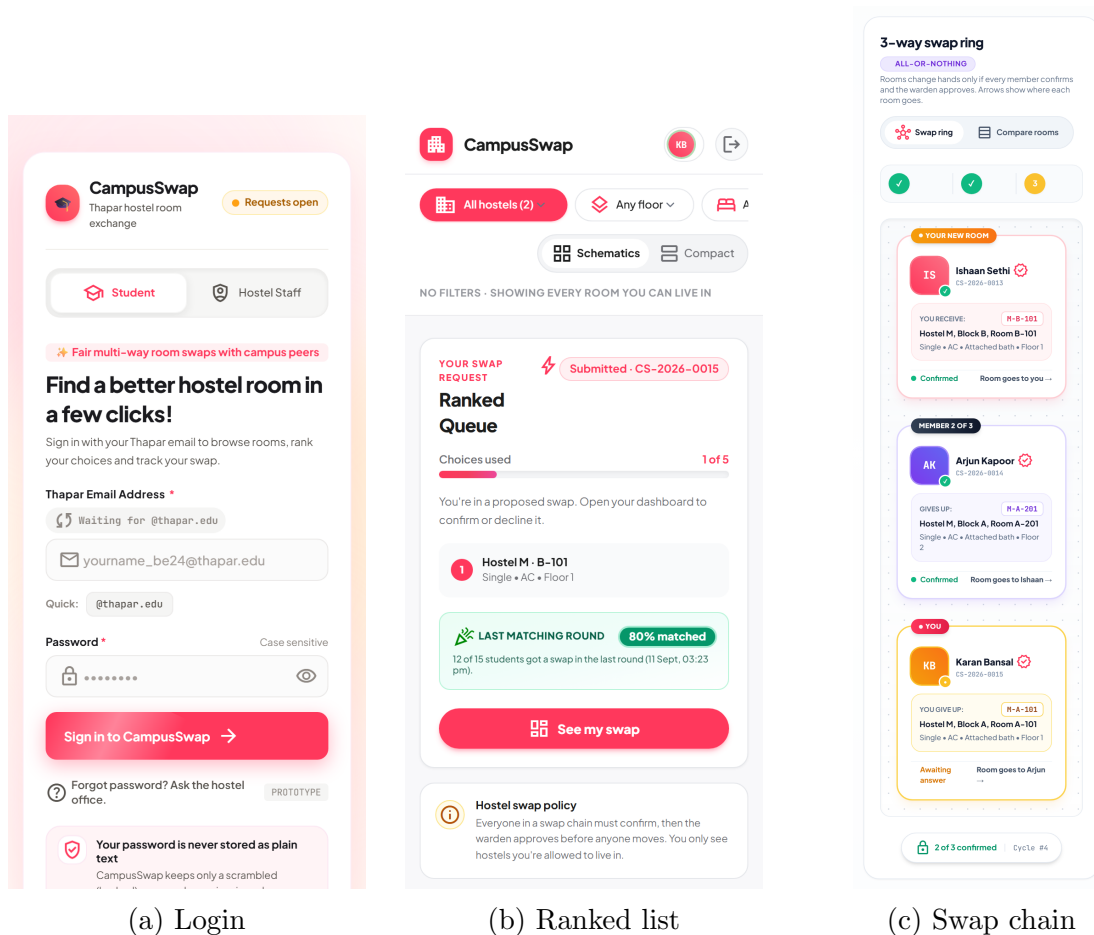


Figure 3.6: The same pages on a phone (390 px wide).

3.5 Repository, CI and Documentation

The project follows the course template repository [JSRK26]. Work is done on a `prototype` branch and merged into `master` through pull requests. A GitHub Actions workflow runs all tests on every push and pull request, and the course workflow publishes the MkDocs documentation site, which includes a study guide for each team role and a question bank.

Testing and Evaluation

4.1 Testing

The suite contains **90 automated tests**, all passing locally and in CI. Each test uses a fresh in-memory database and a fixed clock, so results are repeatable.

Area	Tests	What is checked
Engine (TTC, 1:1, metrics, types)	28	2/3/4-way cycles, fall-back to later choices, double rooms, order independence, 50 random pools obey the guarantees, speed on 2 000 students
Security and demo data	9	password hashing, session tokens, seeded data consistency
Request rules	12	every submission and withdrawal rule
Matching service	9	known chains on the seeded pool, dry runs change nothing, one run at a time
Confirmation workflow	8	confirm, decline, deadline expiry, approval moves students
Page data	9	the data each page receives in every state
HTTP API	15	login, permissions, readable errors, a full 3-way swap from request to approval

Table 4.1: Automated tests.

The four pages were additionally driven end to end in a real browser engine (Microsoft Edge in headless mode): three students submitted, the warden compared and ran a round, the chain was confirmed and approved, and the pages reported no script errors.

4.2 Match Yield

Demo pool. The seeded demonstration pool of 12 requests yields one 2-way, one 3-way and one 4-way chain: $MY = 75.0\%$, against 16.7% for 1:1 swaps only. Adding the three live-demo students (a 3-way chain) gives 12 of 15 matched, $MY = 80.0\%$, against 13.3% for 1:1 swaps. Both results are asserted by automated tests.

Pools of 100 requests. As the proposal specifies 100 requests, we generated 200 synthetic pools of 100 students, each owning one seat and ranking k other seats chosen uniformly at random.

	3 choices each	5 choices each
TTC Match Yield, mean (s.d.)	60.9% (5.6)	75.2% (4.5)
TTC Match Yield, range	45–77%	63–87%
1:1 swaps only, mean (s.d.)	8.4% (3.9)	21.6% (5.2)
Chains per pool, mean	9.2	11.5
Share of chains that are 2-way	16%	17%
Longest chain observed	32	33

Table 4.2: Match Yield on 200 random pools of 100 students.

TTC meets the target $MY \geq 60\%$ on average in both settings and matches three to seven times more students than direct swaps. Only about one chain in six is a simple 2-way swap, which confirms the proposal’s claim that most feasible exchanges are multi-way.

Two cautions apply. First, uniformly random preferences are a simplification: real students concentrate on a few popular rooms, which would lower both yields, so the pilot study remains necessary. Second, TTC can produce very long chains (up to 33 students here); since one decline cancels a whole chain, long chains are fragile. Limiting chain length or re-running TTC on the remaining students after a decline are candidate improvements.

4.3 Performance

Students	TTC time (best of 3)
100	0.2 ms
1 000	4.8 ms
2 000	15.4 ms
5 000	58.0 ms

Table 4.3: TTC running time with five choices per student (Python 3.12, laptop).

Even 5 000 students are matched in well under a tenth of a second, so running a round inside the web request is adequate for a campus.

4.4 Objectives Review

Objective	Status	Evidence
O1 Data model	Met (partly)	11 tables with integrity constraints; a formal migration tool is not yet used
O2 Student module	Met	login, room browser, ranked requests; 12 rule tests
O3 Matching engine	Met	TTC with 28 engine tests
O4 Dashboards and CI	Met	student dashboard, admin console, tests on every push
O5 Evaluation	Met on synthetic data	MY 75.2% (5 choices) and 60.9% (3 choices) on 100-student pools
Phase 2: confirmation	Delivered early	multi-party confirm/decline with 48-hour deadline
Phase 2: warden portal	Partly	approval, CSV export, audit log; no PDF form yet

Table 4.4: Objectives and their status at the prototype stage.

4.5 Deviations from the Proposal

- **No background worker.** Rounds run inside the web request; the measured running times make a job queue unnecessary at this scale.
- **No scheduler.** Overdue chains are dissolved whenever a round runs or a member answers, instead of by a timer.
- **E-mail and password login** instead of the institute login, which requires university approval.
- **SQLite** rather than a database server, to keep the prototype self-contained; SQLAlchemy allows switching later.

Conclusion and Future Work

5.1 Summary

The prototype meets the Phase 1 success definition: it accepts ranked requests, stores them, runs a Top Trading Cycles match and shows the result to students and staff. It also delivers the multi-party confirmation workflow planned for Phase 2, backed by 90 automated tests and a published documentation site.

5.2 Key Findings

- Multi-way exchange matters: on 100-student pools TTC matched 75.2% of students against 21.6% for direct swaps (five choices).
- Keeping the algorithm separate from web and database code made it straightforward to test thoroughly and to measure.
- Long chains are the main practical risk, because one decline cancels the whole chain.

5.3 Future Work

1. Pilot with 20–40 students from one hostel wing to measure real Match Yield, time to approval and confirmation drop-off.
2. Notifications when a chain is found, and the PDF sign-off form.
3. Handling fragile long chains (length limits or re-matching after a decline).
4. Ranking by room type or floor, not only specific rooms.
5. The faculty slot-exchange module, reusing the same engine.
6. Institute single sign-on and deployment to a hosted server.

5.4 Team Roles

Role	Responsibility
P1 Frontend	the four pages, JavaScript modules and styling build
P2 Backend and login	API routes, sessions, request and confirmation rules
P3 Matching engine	TTC, the 1:1 baseline, Match Yield and engine tests
P4 Data, diagrams and delivery	data model, demo data, UML diagrams, CI, documentation and this report

Table 5.1: Team roles; each member records their role in their weekly journal.

Bibliography

- [AS99] Atila Abdulkadiroğlu and Tayfun Sönmez. “House Allocation with Existing Tenants”. In: *Journal of Economic Theory* 88.2 (1999), pp. 233–260 (cit. on p. 5).
- [JSRK26] Jahnavi, Ronit Saini, Ranvir, and Krishna Kapoor. *CampusSwap Source Code and Documentation*. 2026. URL: <https://github.com/ranvir7123/UCS503P-202627-CampusSwap> (cit. on p. 19).
- [Pla26] PlantUML Team. *PlantUML Language Reference Guide*. 2026. URL: <https://plantuml.com/> (cit. on p. 9).
- [Ram26] Sebastián Ramírez. *FastAPI Documentation*. 2026. URL: <https://fastapi.tiangolo.com/> (cit. on p. 9).
- [Rot82] Alvin E. Roth. “Incentive Compatibility in a Market with Indivisible Goods”. In: *Economics Letters* 9.2 (1982), pp. 127–132 (cit. on p. 14).
- [RP77] Alvin E. Roth and Andrew Postlewaite. “Weak versus Strong Domination in a Market with Indivisible Goods”. In: *Journal of Mathematical Economics* 4.2 (1977), pp. 131–137 (cit. on p. 14).
- [SS74] Lloyd Shapley and Herbert Scarf. “On Cores and Indivisibility”. In: *Journal of Mathematical Economics* 1.1 (1974), pp. 23–37 (cit. on pp. 5, 14).
- [SQL26] SQLAlchemy Authors. *SQLAlchemy 2.0 Documentation*. 2026. URL: <https://docs.sqlalchemy.org/en/20/> (cit. on p. 9).